

AI 2002: Artificial Intelligence

Adversarial Search

Spring 2026

[These slides (mostly) are based on those created by Dan Klein and Pieter Abbeel (ai.berkeley.edu).]

[Some slides are by Mr. Sandesh Kumar, and Ms. Anaum Hamid, FAST-NUCES, Karahi]

Adversarial Search

- There might be some situations where more than one agent is searching for the solution in the same search space, and this situation usually occurs in game playing.
- The environment with more than one agent is termed as **multi-agent environment**
- **Search in which two or more players with conflicting goals** are trying to explore the same search space for the solution is called as an **Adversarial search**, often known as **Game Theory**.

Types of Games

- Many different kinds of games!
- Axes:
 - Deterministic or stochastic?
 - One, two, or more players?
 - Zero sum?
 - Perfect information (can you see the state)?
- Want algorithms for calculating a **strategy (policy)** which recommends a move from each state

Deterministic Games

- Many possible formalizations, one is:
 - States: S (start at s_0)
 - Players: $P=\{1\dots N\}$ (usually **take turns**)
 - Actions: A (may depend on player / state)
 - Transition Function: $S \times A \rightarrow S$
 - Terminal Test: $S \rightarrow \{t, f\}$
 - Terminal Utilities: $S \times P \rightarrow R$
- Solution for a player is a **policy**: $S \rightarrow A$

Zero-Sum Games

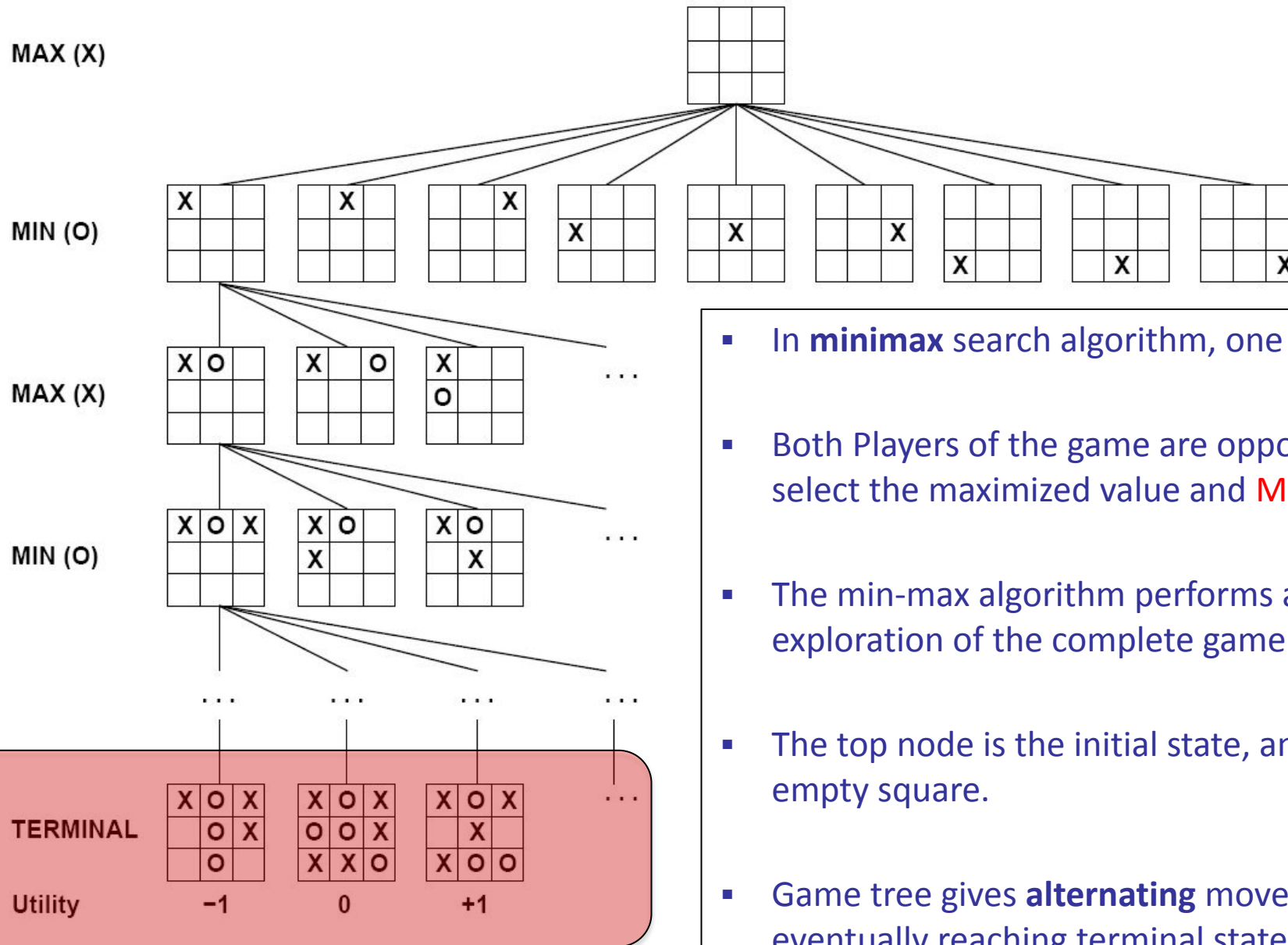
- Zero-Sum Games

- Agents have **opposite** utilities (values on outcomes)
- Lets us think of a single value that one **maximizes** and the other **minimizes**
- Adversarial, pure competition

- General Games

- Agents have independent utilities (values on outcomes)
- Cooperation, indifference, competition, and more are all possible

Tic-Tac-Toe Game Tree (Min-Max Algorithm)

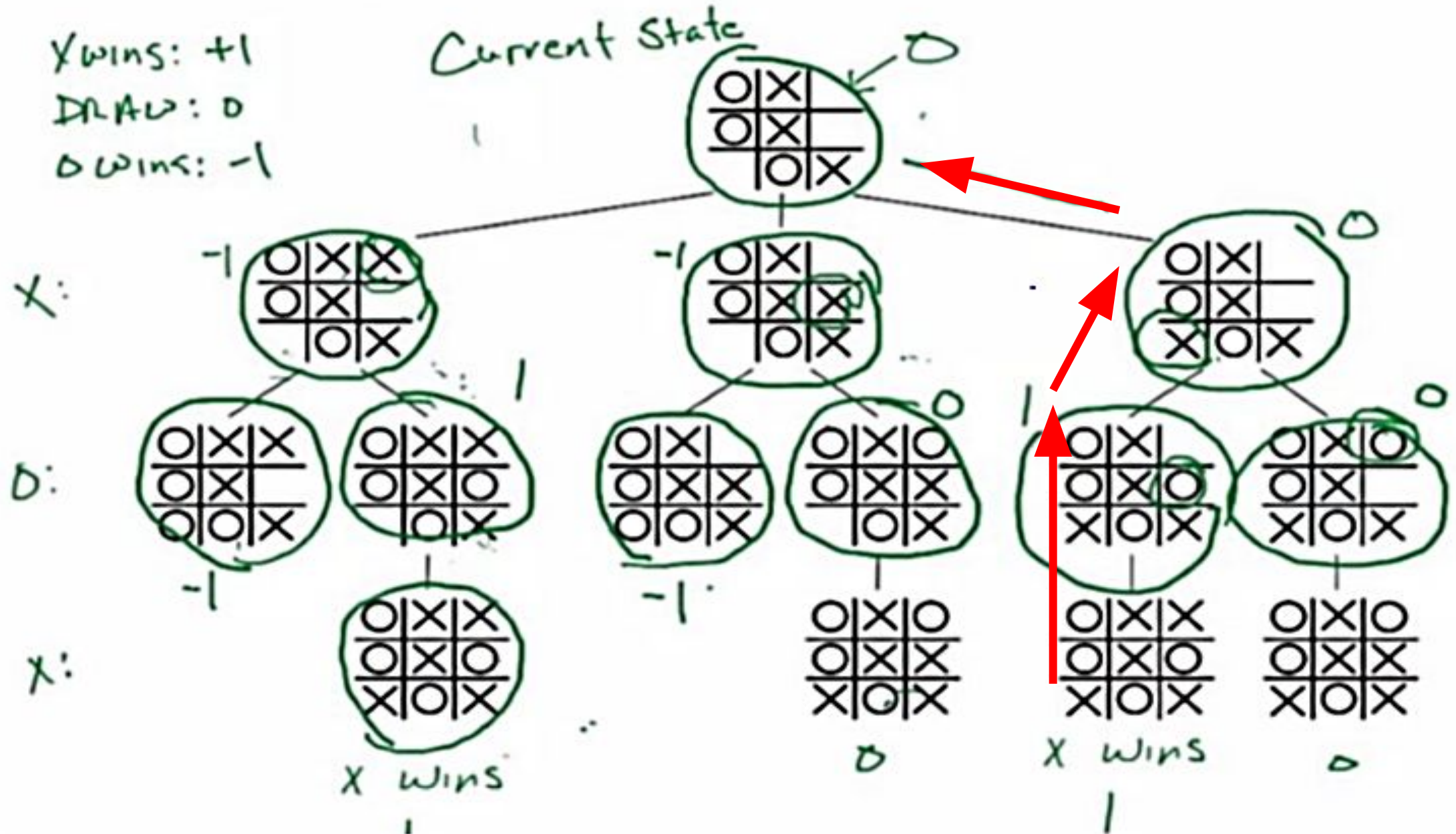


- In **minimax** search algorithm, one is called **MAX** and other is called **MIN**.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and **MIN** will select the **minimized value**.
- The min-max algorithm performs a **depth-first search** algorithm for the exploration of the complete game tree.
- The top node is the initial state, and **MAX** moves first, placing an X in an empty square.
- Game tree gives **alternating** moves by MIN (O) and MAX (X), until eventually reaching terminal states.

Tic-Tac-Toe

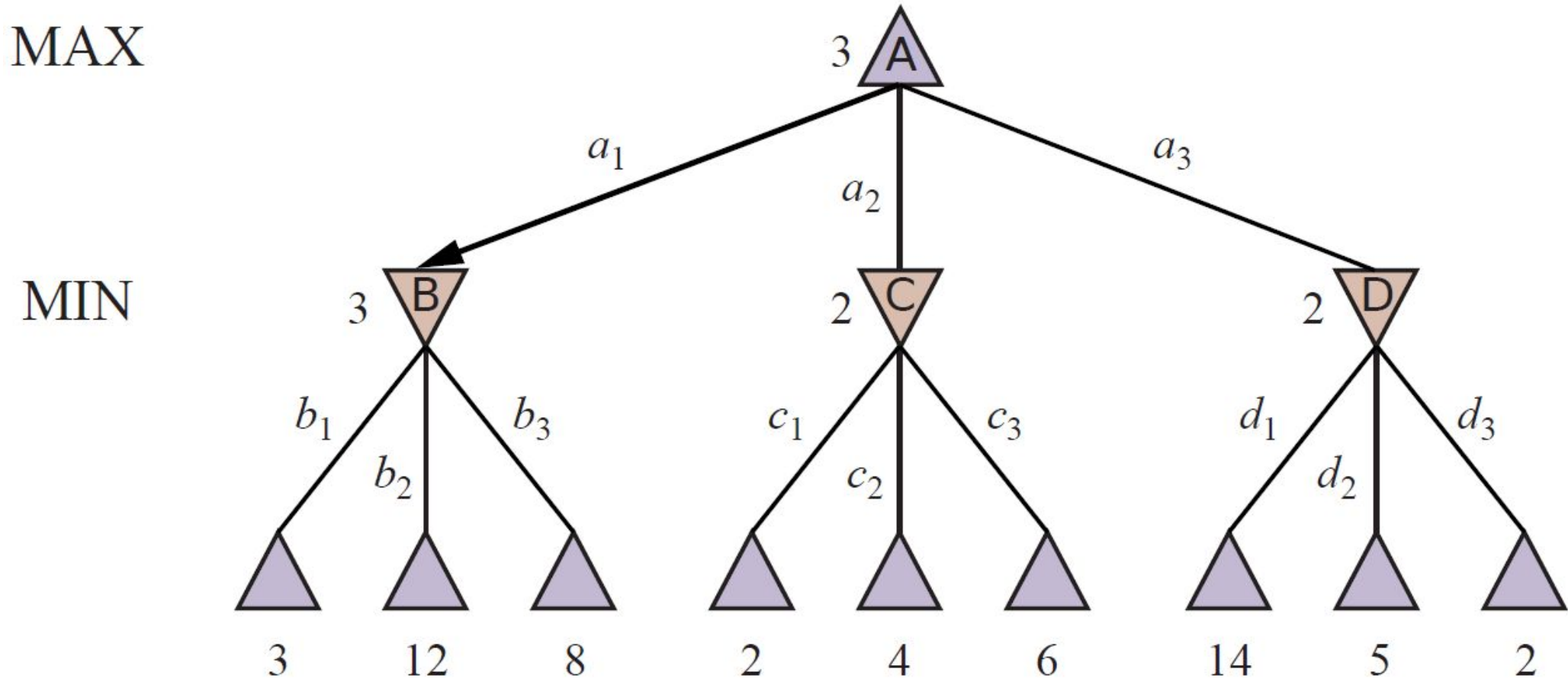
X wins: +1
DRAW: 0
O wins: -1

Current State

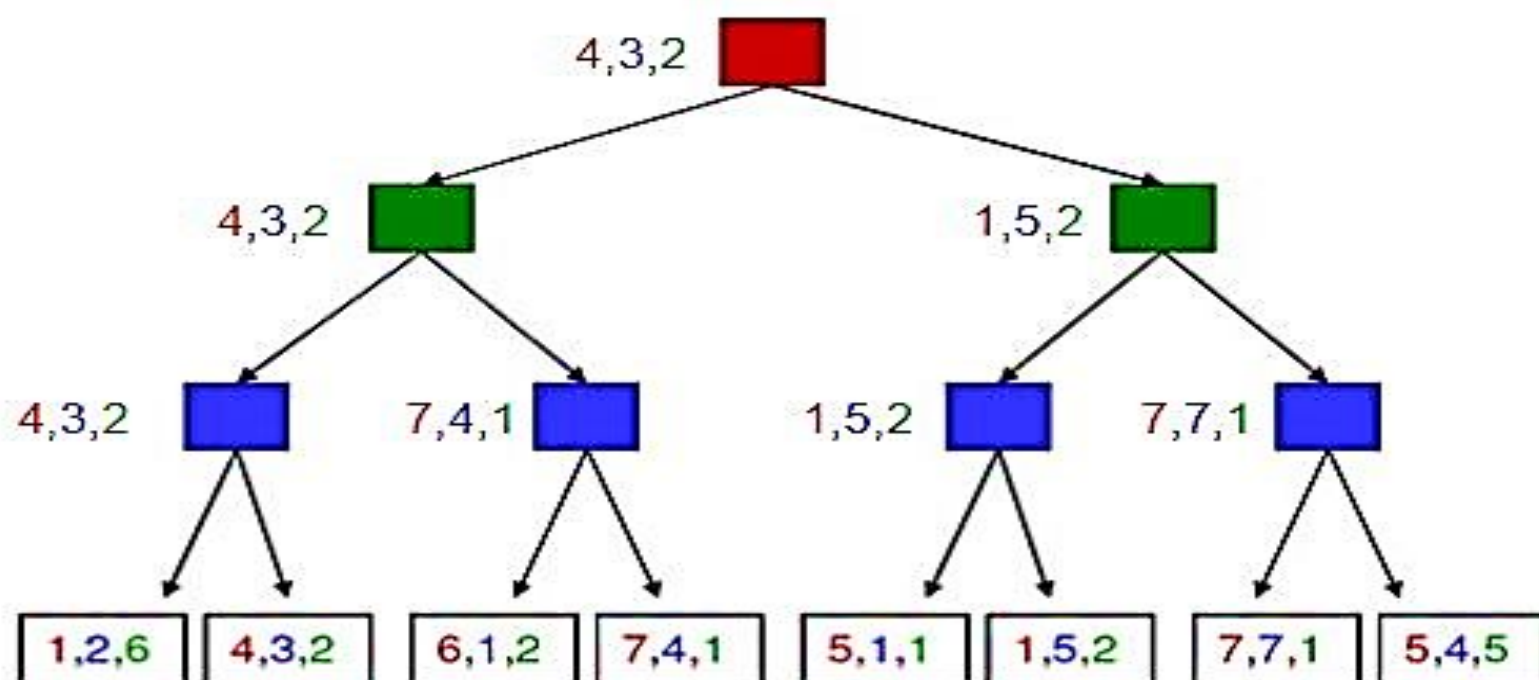


MINIMAX

- Δ nodes are “MAX nodes,” in which it is MAX’s turn to move, and ∇ nodes are “MIN nodes.”
- Terminal nodes show the **utility values** for MAX; other nodes are labeled with their minimax values.
- MAX’s best move at the root is a_1 , because it leads to the state with the highest minimax value, and
- MIN’s best reply is b_1 , because it leads to the state with the lowest minimax value.



Multi-player, non-zero-sum games



- Utilities are tuples
- Each player maximizes their own utility at each node
- Utilities get propagated (*backed up*) from children to parents

Minimax Efficiency

- How efficient is minimax?
 - Just like (exhaustive) DFS
 - Time: $O(b^m)$
 - Space: $O(bm)$
- Example: For chess, $b \approx 35$, $m \approx 100$
 - Exact solution is completely infeasible
 - But, do we need to explore the whole tree?

Minimax Algorithm

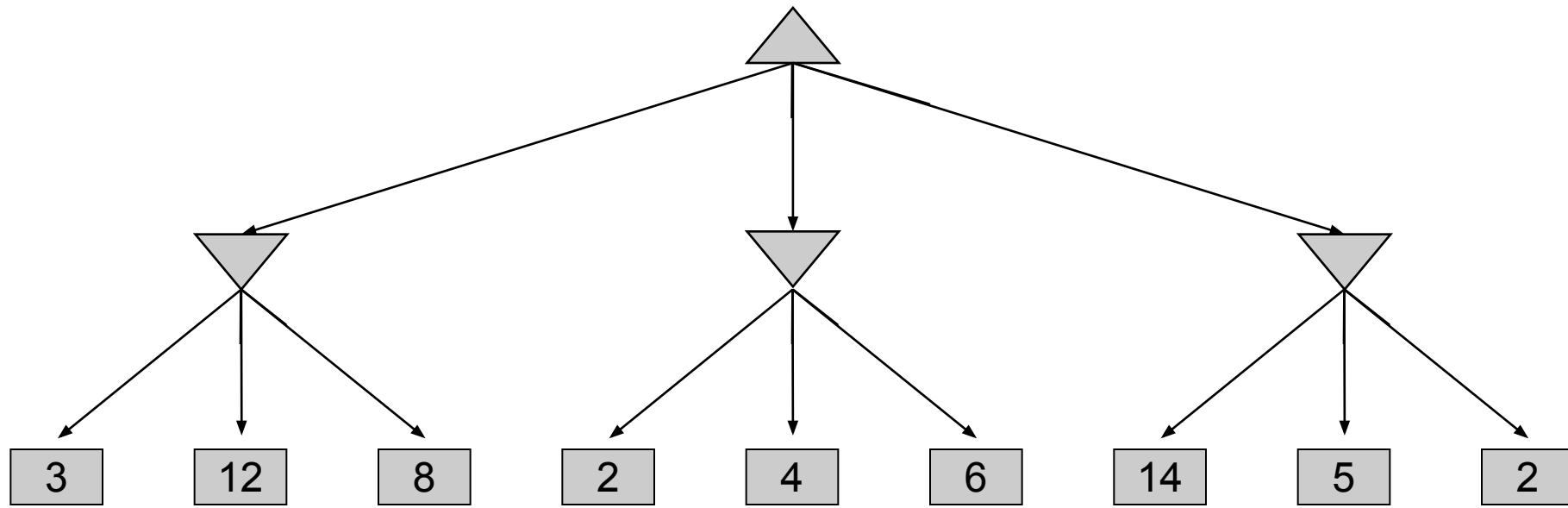
function MINIMAX-DECISION(*state*) *returns an action*
return $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(s, a))$

function MAX-VALUE(*state*) *returns a utility value*
if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 $v \leftarrow -\infty$
for each *a* **in** ACTIONS(*state*) **do**
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$
return *v*

function MIN-VALUE(*state*) *returns a utility value*
if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 $v \leftarrow \infty$
for each *a* **in** ACTIONS(*state*) **do**
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$
return *v*

- Minimax Algorithm returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility.
- The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state.

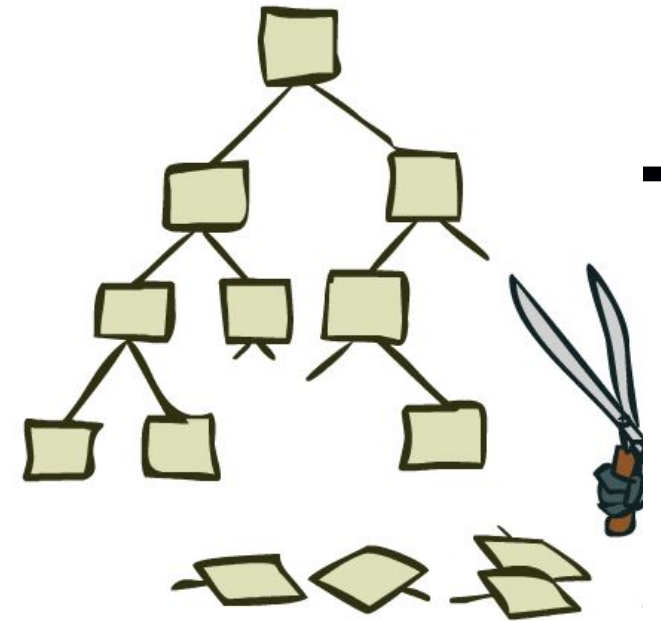
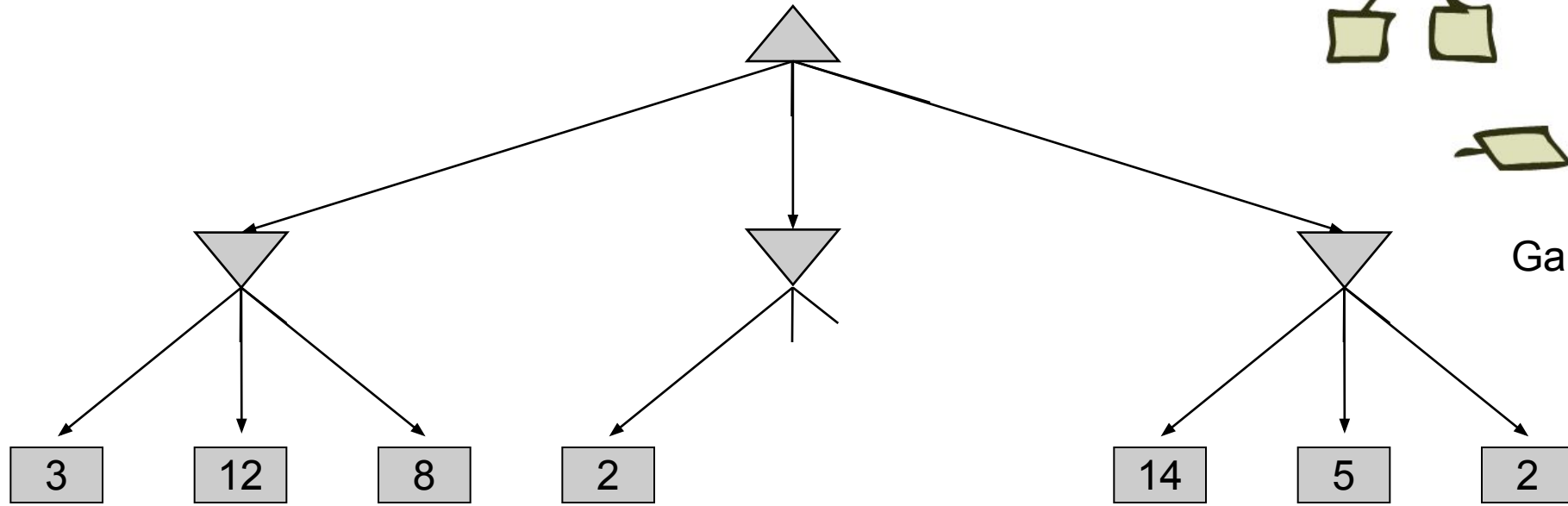
Minimax Example



For MAX Nodes, $V = \max(\alpha)$ & $\alpha \geq -\infty$
For MIN Nodes, $V = \min(\beta)$ & $\beta \leq \infty$

Minimax Pruning

Check at each node
 $\alpha \geq \beta$ (Cut-off)

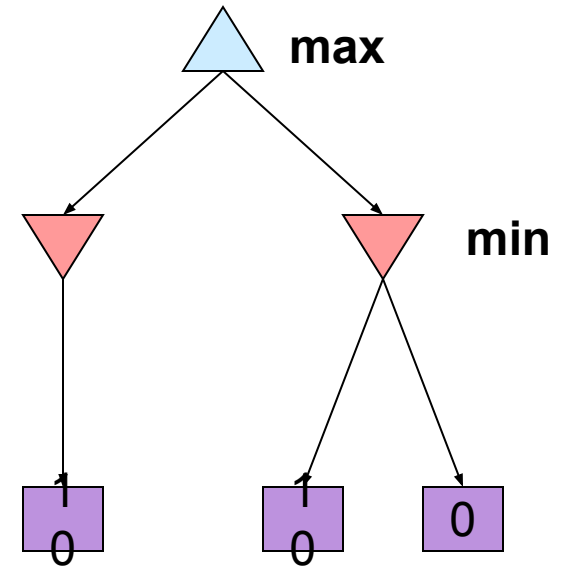


Game Tree Pruning

- It is possible to compute **correct minimax decision** without looking at every node in game tree.
- Pruning: ways of determining that certain branches will not be useful.

Alpha-Beta Pruning Properties

- This pruning has **no effect** on minimax value computed for the root!
- Good child ordering improves effectiveness of pruning
- With “perfect ordering”:
 - Time complexity drops to $O(b^{m/2})$
 - Doubles solvable depth!
 - Full search of, e.g. chess, is still hopeless...



- This is a simple example of **metareasoning** (computing about what to compute)

Alpha-Beta Implementation

α : MAX's best option on path to root
 β : MIN's best option on path to root

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \geq \beta$  return  $v$   
         $\alpha = \max(\alpha, v)$   
    return  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \leq \alpha$  return  $v$   
         $\beta = \min(\beta, v)$   
    return  $v$ 
```

-
- Backup

Games

- In **multiagent** environments, each agent needs to consider the actions of other agents and how they affect its own welfare.
- The agents can be **cooperative** or **competitive**.
- In competitive environments, the agents' goals are in conflict, **giving rise to Adversarial Search problems — often known as GAMES**.
- In mathematical **game theory**, a multiagent environment is treated as a game, the impact of each agent (economy) on the others is significant, regardless of whether the agents are cooperative or competitive.
- In AI, the most common games are deterministic, turn-taking, two-player, zero-sum games of perfect information (such as chess).
 - In deterministic and fully observable environments in two agents act alternately and in which the utility values at the end of the game are always equal and opposite.
 - If one player wins a game of chess, the other player necessarily loses.
 - It is this opposition between the agents' utility functions that makes the situation adversarial.

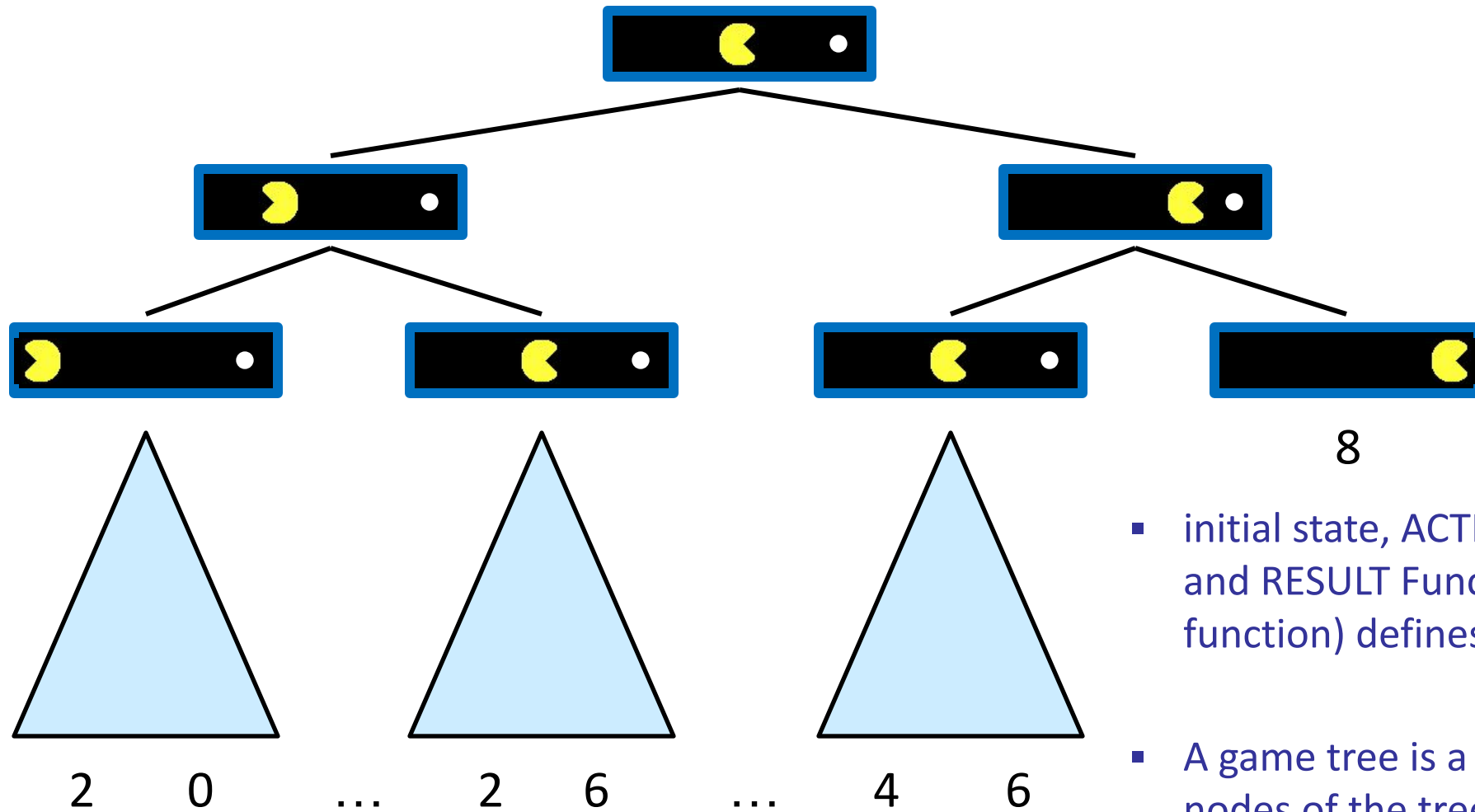
Games

- Games are too hard to solve.
- Chess has an average branching factor of about 35, and games often go to 50 moves by each player, so the search tree has about 35^{50} nodes.
- Games require the ability to make decision even when calculating optimal decision is infeasible.
- **Pruning** allows us to ignore portions of the search tree.
- **Heuristic evaluation functions** allow us to approximate true utility of a state **without doing a complete search**.
- Games such as poker includes elements of **imperfect information** because not all cards are visible to each player.

Types of games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleships,	bridge, poker,

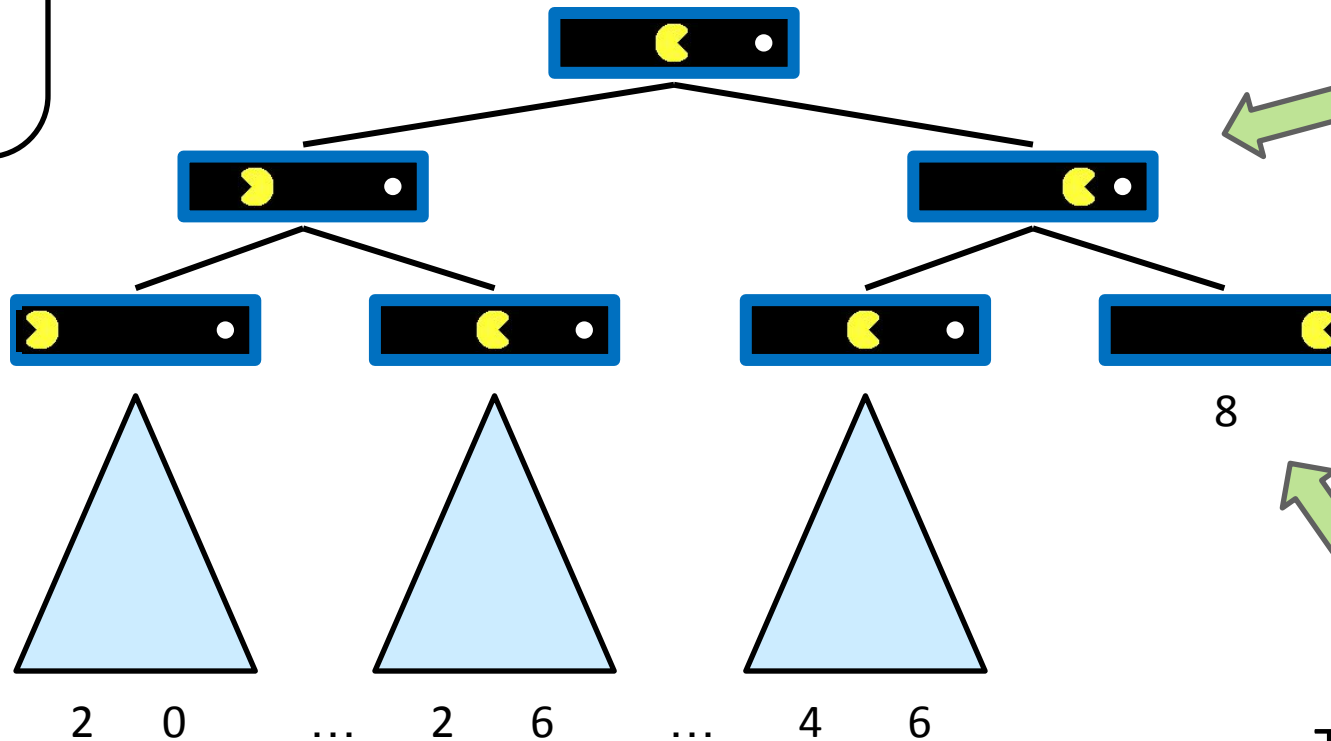
Single-Agent Trees



- initial state, ACTIONS function, and RESULT Function (utility function) defines the game tree
- A game tree is a tree where nodes of the tree are the game states and Edges of the tree are the moves by players.

Value of a State

Value of a state:
The best achievable
outcome (utility)
from that state



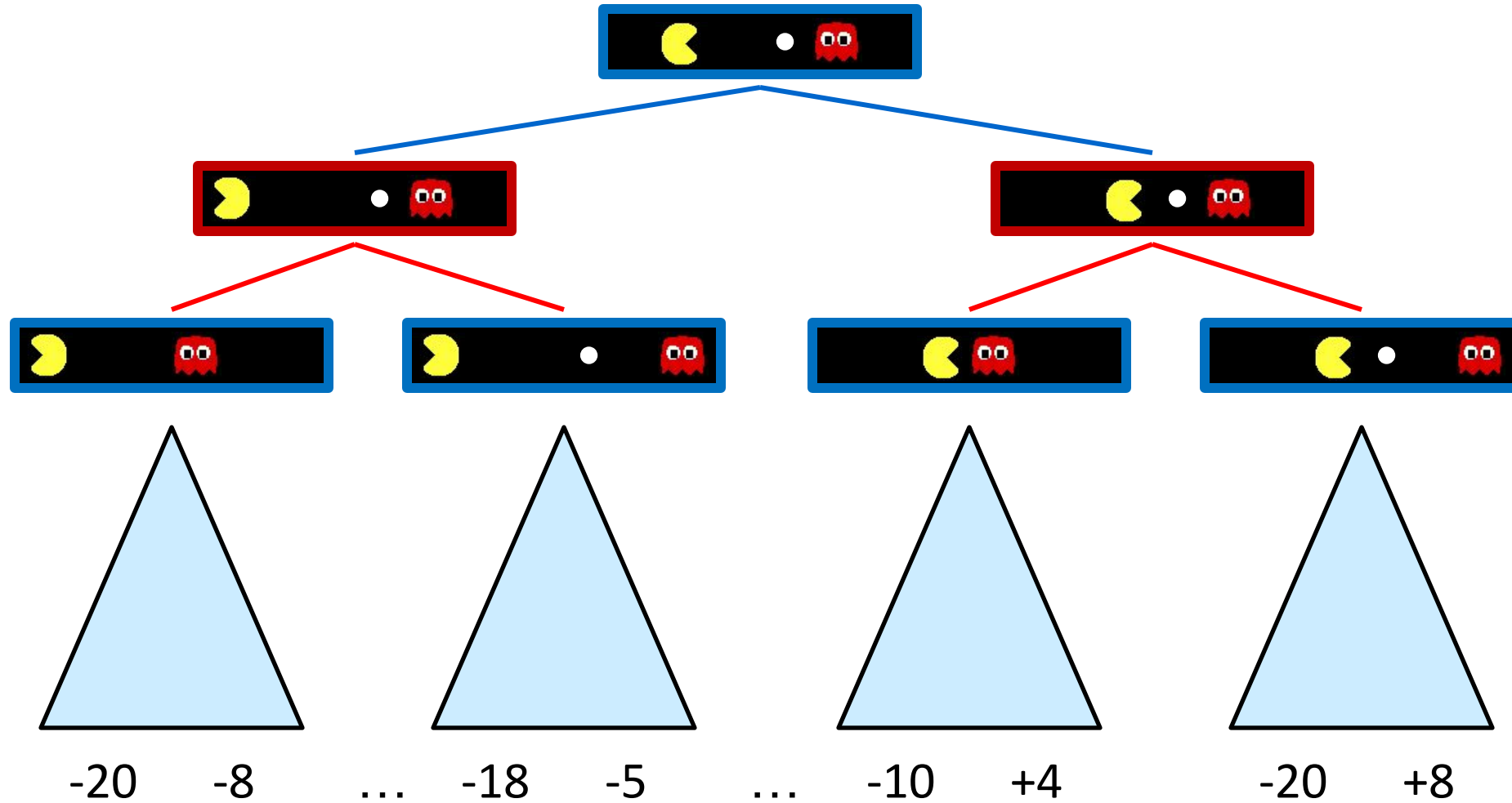
Non-Terminal States:

$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

Terminal States:

$$V(s) = \text{known}$$

Adversarial Game Trees



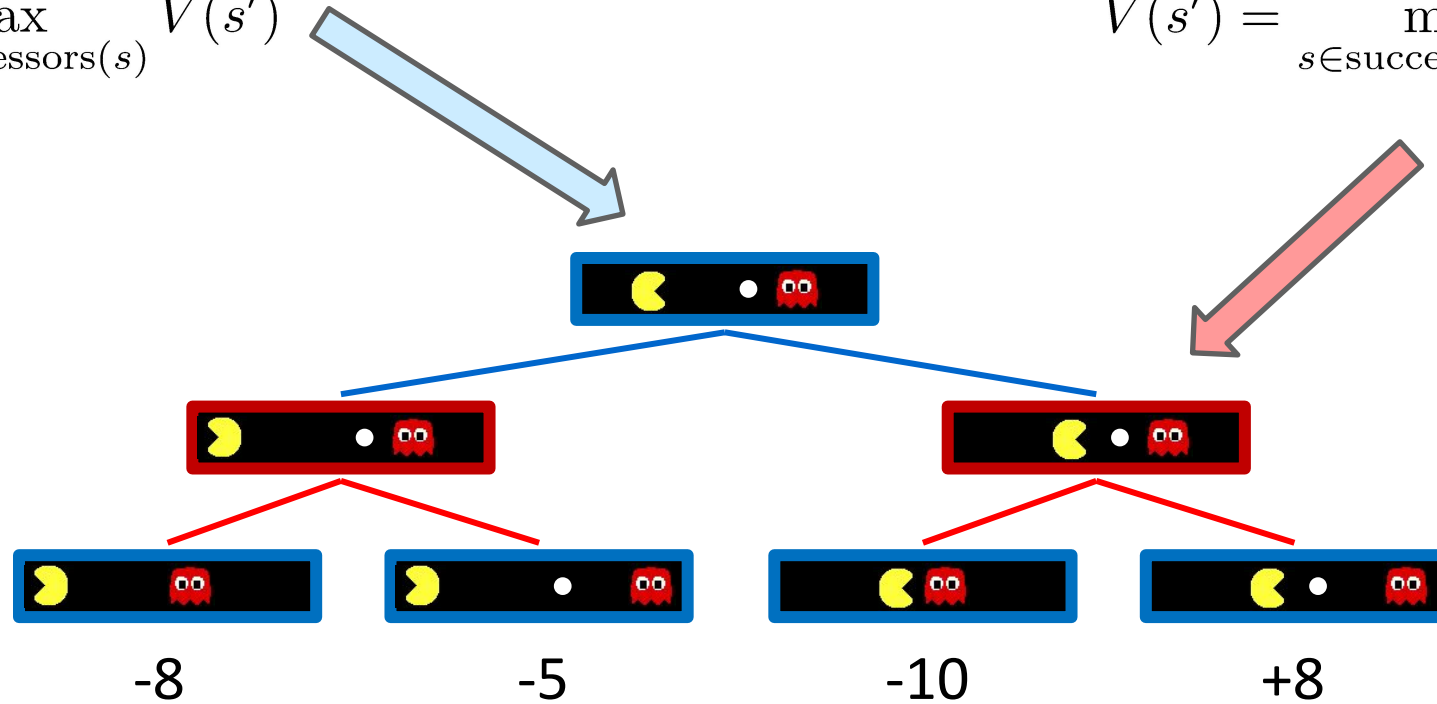
Minimax Values

States Under Agent's Control:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

States Under Opponent's Control:

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$



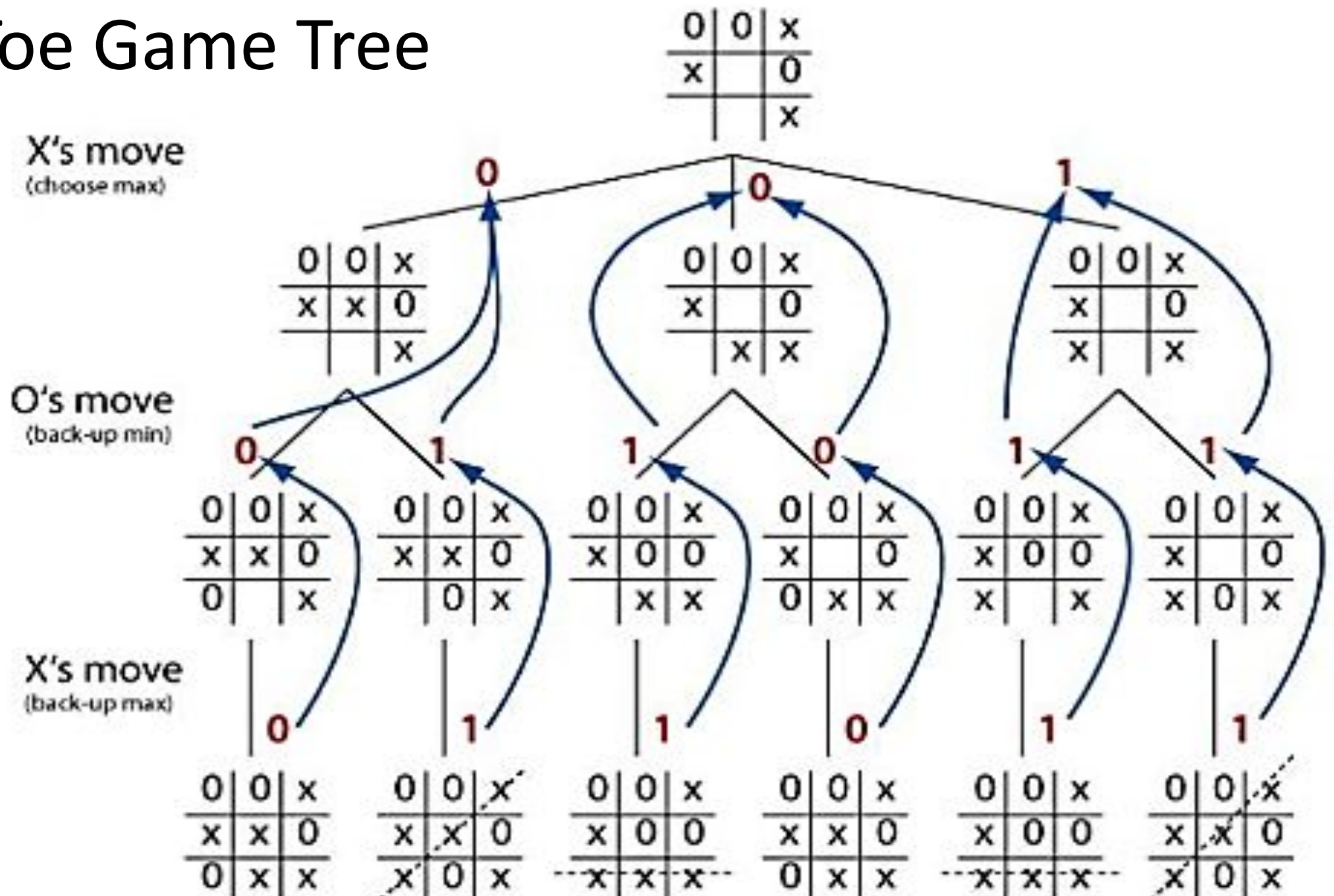
Terminal States:

$$V(s) = \text{known}$$

Min-Max Algorithm

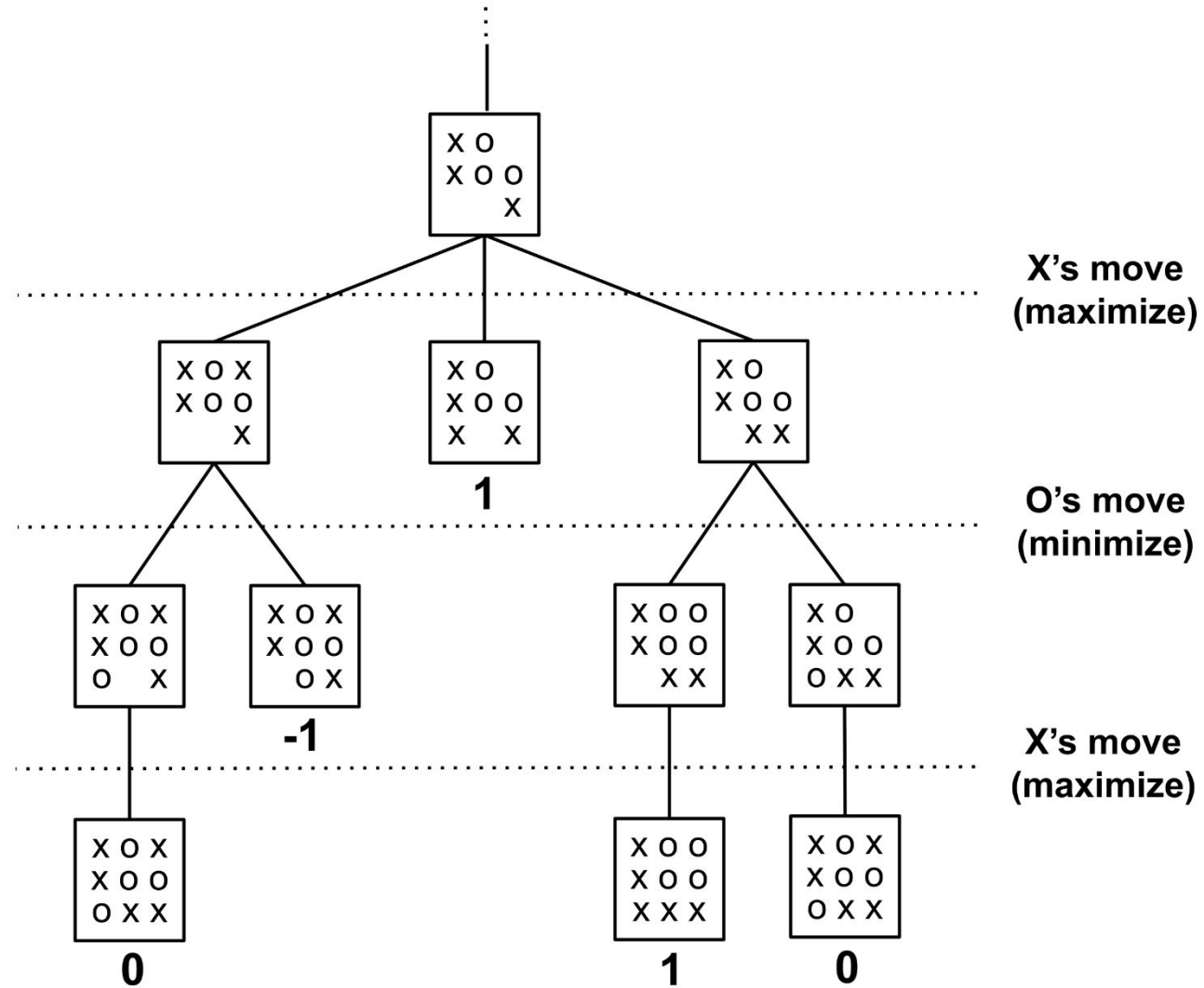
- In this algorithm two players play the game, one is called MAX and other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The min-max algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
- The min-max algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Tic-Tac-Toe Game Tree



Minimax Values

from perspective of player X



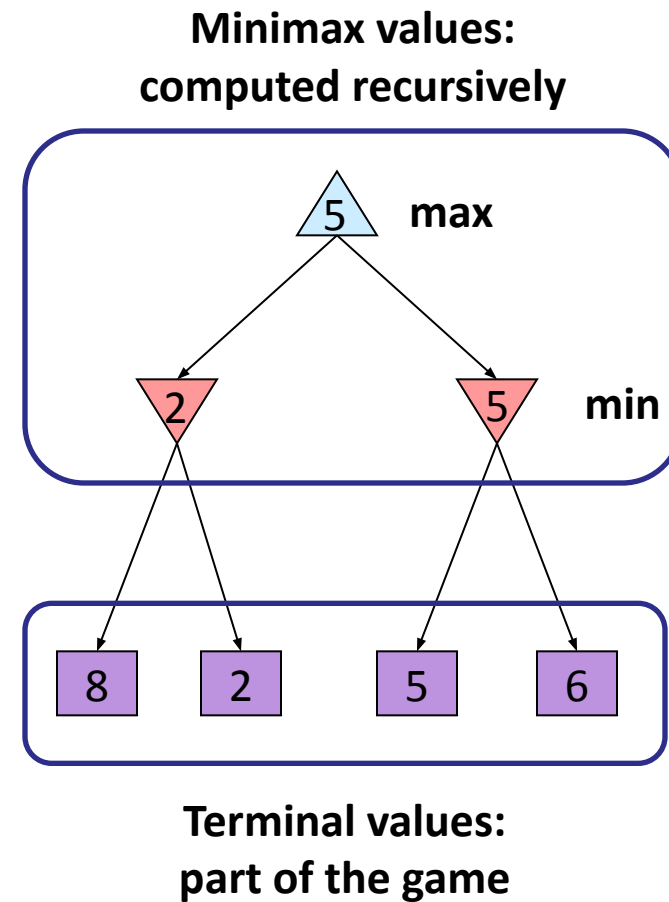
Optimal Decisions in Games

- **Optimal Solution** in search problem is **sequence of actions leading to** a goal state (terminal state that is a **win**)
- MAX must find a contingent strategy, which specifies MAX's move in the initial state, **then MAX's moves in the states resulting from every possible response by MIN.**
- Given a game tree, optimal strategy can be determined from minimax value of each node n , and it is called as MINIMAX(n).
- The minimax value of a node is the utility (for MAX) of being in the corresponding state, assuming that **both players play optimally** from there to the end of the game.
- The **minimax value of a terminal state is just its utility.**
- MAX prefers to move to a state of maximum value, whereas MIN prefers a state of minimum value.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

Adversarial Search (Minimax)

- **Deterministic, zero-sum games:**
 - Tic-tac-toe, chess, checkers
 - One player maximizes result
 - The other minimizes result
- **Minimax search:**
 - A state-space search tree
 - Players alternate turns
 - Compute each node's **minimax value**: the best achievable utility against a rational (optimal) adversary



Minimax Implementation (Dispatch)

```
def value(state):
```

if the state is a terminal state: return the state's utility

if the next agent is MAX: return max-value(state)

if the next agent is MIN: return min-value(state)

```
def max-value(state):
```

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}))$

return v

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

```
def min-value(state):
```

initialize $v = +\infty$

for each successor of state:

$v = \min(v, \text{value}(\text{successor}))$

return v

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$